



BOOK FOUR

Code Glossary

Every keyword and symbol in plain language, with a real RuWork example

A quick-reference cheat sheet. Every entry is explained in plain language with a real RuWork example. Use it when you hit a keyword or symbol you don't recognise.

CONTENTS



1	JavaScript	1
2	Node & Express	7
3	MongoDB & Mongoose	9
4	Authentication & security	12
5	React	16
6	HTTP	19
7	RuWork-specific terms	21

JavaScript

const

Declares a variable that cannot be **reassigned**. The default choice in RuWork.

```
const email = normalizeEmail(dataU.email);
email = "other@x.lk"; // X TypeError
```

JS

Careful: `const` prevents reassignment, not mutation. `const user = {}; user.name = "A";` is legal — you changed the object's contents, not which object the name points to.

let

Declares a variable you intend to reassign. Used sparingly — mostly for accumulating values.

```
let review; // controllers/reviewController.js
review = new Review({ ... }); // assigned later inside try
```

JS

Arrow functions

A shorter function syntax. $(a, b) \Rightarrow a + b$ is roughly `function (a, b) { return a + b; }`.

```
jobs.map((job) => serializeJobSummary(job))
```

JS

A single expression body returns implicitly (no `return` needed). Wrapping an object literal in parentheses is required, otherwise `{` is read as a code block:

```
setForm((current) => ({ ...current, email: value })); // parentheses matter
```

JS

Callbacks

A function passed to another function to be run later.

```
app.listen(getPort(), () => { logger.info("RuWork API started", ...); });
```

JS

The arrow function is a callback — Express calls it once the server is listening.

Promises

An object representing a value that isn't ready yet. It is *pending*, then either *fulfilled* or *rejected*.

```
adminService.getAudits(params)
  .then((data) => setState({ status: "success", ... }))
  .catch((error) => setState({ status: "error", ... }));
```

JS

async / await

Syntax that makes promise code read top-to-bottom. `async` marks a function as returning a promise; `await` pauses until a promise settles.

```
export async function loginUser(req, res) {
  const user = await User.findOne({ email });
  const ok = await bcrypt.compare(dataU.password, user.password);
}
```

JS

Why it matters: *await* does **not** block the server. While this request waits on MongoDB, Node serves other requests. It just makes your code wait.

try / catch / finally

Error handling. Code in `try` runs; if it throws, `catch` receives the error; `finally` runs either way.

```
const original = User.findById;
User.findById = async () => account;
try { /* exercise the guard */ }
finally { User.findById = original; } // restored even if the assertion fails
```

JS

throw

Raises an error, stopping normal execution until a catch.

```
throw new AdminInputError(`${unexpected} cannot be changed through this Admin operation`);
```

JS

RuWork throws **custom error classes** (`AdminInputError`, `ApplicationConflictError`, `ReviewInputError`) so a single catch can map each type to the right HTTP status.

Destructuring

Pulling values out of an object or array into named variables.

```
const { page, limit } = adminPagination(req.query); // object
const [scheme, token] = authorizationHeader.split(" "); // array
const { data } = await api.get("/admin/jobs"); // very common in services
```

Also works in parameters, with defaults:

```
export default function Spinner({ label = "Loading RuWork" }) { ... }
```

Spread ...

Expands an object or array.

```
const filter = { ...(status === "all" ? {} : { status }) }; // conditionally include a key
setForm((current) => ({ ...current, email: value })); // copy then override
export const SETTINGS_DEFAULTS = { ..., ...settings }; // merge, later wins
```

In a parameter list it means “collect the rest”:

```
export function authorizeRoles(...allowedRoles) { ... }
isAdmin = authorizeRoles(ADMIN_ROLE);
```

Optional chaining ?.

Safely reads a property that might not exist. Returns `undefined` instead of throwing.

```
req.user?.sub // no crash if req.user is undefined
error?.response?.data?.code
value?.toString?.() // also guards the method itself
```

Without it, `req.user.sub` on an undefined `req.user` throws “*Cannot read properties of undefined*” — the most common JavaScript crash.

Nullish coalescing ??

Returns the right side only when the left is `null` or `undefined`.

```
settings.studentRegistrationOpen ?? SETTINGS_DEFAULTS.studentRegistrationOpen
```

?? vs ||: `||` also triggers on `0`, `""`, and `false`. For a boolean setting, `false || true` wrongly gives `true`; `false ?? true` correctly gives `false`. Choose `??` whenever `0`, `""`, or `false` are valid values.

Template literals

Backtick strings with `${}` interpolation and multi-line support.

```
`${clientId}/verify-email?${query.toString()}`  
`Registration ${decision} successfully`
```

JS

map

Transforms every element into a new array of the same length.

```
jobs.map((job) => serializeJobSummary(job))
```

JS

filter

Keeps only the elements passing a test.

```
registrations.filter((item) => item.accountStatus === "pending")
```

JS

find

Returns the **first** matching element, or undefined.

```
const unexpected = Object.keys(body).find((field) => !allowed.includes(field));  
if (unexpected) throw new AdminInputError(`{unexpected} cannot be changed...`);
```

JS

some / every

some → true if **any** element passes. every → true if **all** do.

```
if (getCorsOrigins().some((origin) => origin === "*")) problems.push("...wildcard...");  
skills.every((skill) => skill.length ≤ 50)
```

JS

includes

Is this value in the array (or substring in the string)?

```
if (!ACCOUNT_STATUSES.includes(status)) return { error: "Invalid registration status" };
```

JS

RuWork uses this constantly for **allowlists** — the safest way to validate input against a fixed set.

reduce

Collapses an array into a single value.

```
const total = results.reduce((sum, result) => sum + result.total, 0);
```

JS

Set

A collection of unique values with fast membership checks.

```
const NON_SESSION_401_CODES = new Set(["CURRENT_PASSWORD_INVALID"]);
if (!NON_SESSION_401_CODES.has(code)) { ... }

return [...new Set(origins)]; // dedupe an array
```

JS

Object.hasOwn / Object.keys / Object.entries / Object.fromEntries

```
Object.hasOwn(req.body, "status") // does the body have this key?
Object.keys(SETTINGS_DEFAULTS)    // ["studentRegistrationOpen", ...]
Object.entries(statistics.jobs)    // [["total", 9], ["draft", 2], ...]
Object.fromEntries(fields.map((f) => [f, ...])) // build an object from pairs
```

JS

Object.freeze

Makes an object immutable at runtime.

```
export const SETTINGS_DEFAULTS = Object.freeze({ studentRegistrationOpen: true, ... });
```

JS

Prevents accidental mutation of a shared constant.

Modules — import / export

RuWork uses **ES modules** everywhere ("type": "module" in both package.json files).

Named exports — many per file, imported by exact name:

```
export function normalizeEmail(value) { ... }
import { normalizeEmail, createAccessToken } from "../utils/account.js";
```

JS

Default export — one per file, named freely on import:

```
const User = mongoose.model("User", userSchema);
export default User;

import User from "../models/user.js";
```

JS

Backend gotcha: ES modules require the `.js` extension in relative imports. `from "../models/user"` fails in Node. Vite (frontend) resolves extensions for you, which is why frontend imports omit them.

`crypto.randomBytes`

Cryptographically secure random bytes.

```
const rawToken = crypto.randomBytes(32).toString("hex"); // 64 hex characters
```

JS

Never use `Math.random()` for anything security-related. It is predictable — an attacker who observes a few outputs can compute the rest, and forge verification or reset tokens.

Node & Express

Middleware

A function (`req, res, next`) in the request pipeline. It either passes control on (`next()`) or ends the request (`res.status(...).json(...)`).

```
export function authenticateToken(req, res, next) {  
  if (bad) return res.status(401).json({ error: "Authentication is required" }); // stop  
  req.user = jwt.verify(token, process.env.JWT_SECRET);  
  return next(); // continue  
}
```

JS

Router

A mini-application for a group of related routes, mounted under a prefix.

```
const userRouter = express.Router();  
userRouter.post("/login", authRateLimiter, loginUser);  
// mounted in index.js:  
app.use("/api/users", userRouter); // → POST /api/users/login
```

JS

Controller

The final handler that produces the response. Validates input, calls models/utils, sends JSON.

Model

A Mongoose model — the programmatic interface to one MongoDB collection (`User`, `Job`, `Application`).

Utility

Shared logic in `utils/` with **no knowledge of req/res** — which is what makes it easy to unit-test.

req

The request. Key properties:

PROPERTY	SOURCE	EXAMPLE
<code>req.params</code>	:placeholders in the path	<code>/api/jobs/:id</code> → <code>req.params.id</code>
<code>req.query</code>	after <code>?</code>	<code>?page=2</code> → <code>req.query.page</code> (a string)
<code>req.body</code>	parsed JSON body	<code>{ email, password }</code>
<code>req.get("Header")</code>	a header	<code>req.get("Authorization")</code>
<code>req.user</code>	RuWork adds this — verified JWT claims	set by <code>authenticateToken</code>
<code>req.studentAccount</code>	RuWork adds this — live DB document	set by <code>requireEligibleRuhunaStudent</code>

res

The response.

```
return res.status(201).json({ message: "Job published successfully", job });
```

JS

Send **exactly once** — hence the `return` in front of every response in RuWork.

next

Passes control to the next middleware. `next(error)` jumps straight to the error handler.

Route parameter

A named path segment: `/api/jobs/:id`. Read via `req.params.id`.

Query parameter

Key/value pairs after `?`: `/api/jobs?category=Tutoring&page=2`. Read via `req.query`. **Always strings** — and can arrive as arrays if repeated, which is why `adminPagination` rejects non-scalars.

Body

The JSON payload of a POST/PATCH. Parsed by `express.json()` into `req.body`, capped at 100 kB.

Header

Metadata sent with a request or response. RuWork uses `Authorization`, `Content-Type`, and the `Helmet` security headers.

Bearer token

The scheme for sending a token: `Authorization: Bearer eyJhbGciOi...`. “Bearer” means *whoever holds this token may use it* — which is why tokens must never be logged or put in URLs.

MongoDB & Mongoose

Schema

The blueprint for a document: fields, types, rules, defaults, indexes.

```
const reviewSchema = new mongoose.Schema({  
  rating: { type: Number, required: true, min: 1, max: 5,  
    validate: { validator: Number.isInteger, message: "Rating must be a whole number from 1 to 5" } }  
}, { timestamps: true });
```

JS

Model

A schema compiled into a usable object.

```
const Review = mongoose.model("Review", reviewSchema);
```

JS

Document

One record — one job, one application. Roughly a row.

Collection

A group of documents — jobs, users. Roughly a table.

ObjectId

MongoDB's 12-byte unique identifier, shown as 24 hex characters.

```
if (!mongoose.isValidObjectId(req.params.id)) return res.status(404).json({ error: "Job not found" });
```

JS

Always validate before querying. An invalid id thrown at a query produces a `CastError`; RuWork checks first and returns a clean 404.

Index

A sorted structure that makes lookups fast. Without one, MongoDB scans every document.

```
jobSchema.index({ category: 1, status: 1 }); // 1 = ascending
applicationSchema.index({ jobId: 1, studentId: 1 }, { unique: true }); // also enforces uniqueness
```

JS

enum

Restricts a field to a fixed list.

```
status: { type: String, enum: ["draft", "open", "closed"], default: "open" }
```

JS

immutable

The value can never change after creation.

```
jobProviderId: { type: ObjectId, ref: "JobProvider", required: true, immutable: true }
```

JS

select: false

Excluded from query results by default; must be asked for explicitly.

```
emailVerificationTokenHash: { type: String, select: false }
Model.findOne(q).select("+emailVerificationTokenHash") // explicit opt-in
```

JS

Why: a token hash can never leak into a response by accident.

lean()

Returns plain JavaScript objects instead of Mongoose documents — faster and lighter, but with no `.save()`.

```
Job.find(filter).lean().exec()
```

JS

save()

Persists a document and runs schema validation and hooks.

populate()

Replaces a stored id with the referenced document.

```
.populate({ path: "jobProviderId", select: "companyName industry companyWebsite" })
```

JS

The `select` limits which fields come back — so a provider's email and password hash cannot leak into a public listing.

Query

A database request. Chainable before execution:

```
Model.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit).lean().exec()
```

JS

Aggregation

A pipeline of processing stages run inside the database.

```
Review.aggregate([
  { $match: { jobId, moderationStatus: { $ne: "hidden" } } },
  { $group: { _id: null, averageRating: { $avg: "$rating" }, reviewCount: { $sum: 1 } } }
]);
```

JS

`$match` filters, `$group` computes. Far cheaper than fetching every review and averaging in JavaScript.

Common operators

OPERATOR	MEANING	RUWORK USE
<code>\$ne</code>	not equal	<code>moderationStatus: { \$ne: "hidden" }</code>
<code>\$gt</code>	greater than	<code>applicationDeadline: { \$gt: now }</code>
<code>\$in</code>	in a list	<code>status: { \$in: HISTORY_STATUSES }</code>
<code>\$or</code>	any condition	Admin search across several fields
<code>\$set</code>	set fields in an update	<code>Job.updateMany(..., { \$set: { providerSuspendedAt } })</code>
<code>\$regex</code>	pattern match	Admin search — always escaped first
<code>\$text</code>	full-text search	Public job search

Error 11000

MongoDB's duplicate-key error code, raised when a unique index is violated.

```
if (error?.code === 11000) return res.status(409).json({ error: "An account already uses this email" });
```

JS

This is how RuWork *guarantees* no duplicate emails, applications, or reviews — the database enforces it even under simultaneous requests.

Authentication & security

Hash

A one-way transformation. You can compute it forwards; you cannot reverse it.

Salt

Random data mixed into a hash so identical inputs produce different hashes. Defeats precomputed “rainbow tables”. bcrypt salts automatically.

bcrypt

A deliberately slow, salted password-hashing algorithm.

```
const hashedPassword = await bcrypt.hash(password, 10);    // register
const ok = await bcrypt.compare(submitted, user.password); // login
```

JS

The 10 is the cost factor — 2^{10} rounds, ~100 ms. Slowness is the feature: it makes mass guessing impractical.

JWT (JSON Web Token)

A signed token in three base64url parts: `header.payload.signature`.

The payload is encoded, not encrypted — anyone can read it. The *signature* is what proves it came from your server. Never put a secret in a JWT.

Claim

One field inside the payload. RuWork’s: `sub` (account id), `email`, `firstName`, `lastName`, `role`, `tv` (revocation counter), `exp` (expiry).

Token

Any string that proves something. RuWork has three kinds, all treated as credentials:

TOKEN	PURPOSE	STORED AS
Access token (JWT)	Prove identity per request	Not stored server-side
Verification token	Prove control of an email	SHA-256 hash only
Reset token	Authorise a password reset	SHA-256 hash only

Authentication

Who are you? Failure → **401**.

Authorization

Are you allowed to do this? Failure → **403**.

“

The classic mix-up. 401 = “log in”. 403 = “logged in, still not allowed”.

”

CORS (Cross-Origin Resource Sharing)

The browser rule that a page from origin A cannot read a response from origin B unless B allows it. RuWork uses a strict allowlist from `CORS_ORIGINS/CLIENT_URL`; a wildcard is rejected in production.

Helmet

Middleware that sets protective response headers — CSP, X-Content-Type-Options, Referrer-Policy, frame-ancestors, HSTS in production.

Rate limit

A cap on requests per client per time window. Exceeding it → **429**. RuWork has three tiers; the login limiter counts only *failures*.

XSS (Cross-Site Scripting)

An attacker gets their JavaScript to run on your page, letting them read anything the page can — including `sessionStorage`. RuWork renders message and review content as **plain text**, never as HTML.

CSRF (Cross-Site Request Forgery)

A malicious site makes an authenticated request using cookies the browser sends automatically.

Why RuWork is not currently exposed: *it authenticates with an `Authorization` header, not cookies. Browsers do not attach headers automatically, so a third-party site cannot forge an authenticated request. If RuWork ever moved to cookie sessions, CSRF protection would become mandatory.*

Enumeration attack

Using different responses to discover which accounts exist. RuWork defends with identical responses:

- Login: "Invalid email or password" for both a wrong email and a wrong password.
- Forgot password: one generic body for unknown / suspended / cooling-down / sent.

Brute-force attack

Trying many passwords until one works. Countered by bcrypt's slowness plus the login rate limiter.

Mass assignment

Passing a whole request body into a model, letting a client set fields it should not.

```
new User(req.body) // ✗ dangerous
new User({ firstName: ..., role: STUDENT_ROLE }) // ✓ RuWork builds explicitly
```

JS

Allowlist (whitelist)

Permit only known-good values; reject everything else. The opposite of a denylist, and much safer — you cannot forget to block something you never thought of. Used for settings fields, sort options, filters, and admin request bodies.

ReDoS (Regular-expression Denial of Service)

A crafted pattern that makes a regex take exponential time, pinning a CPU. Prevented by `escapeAdminRegex()`, which turns user input into a literal string.

Secret

A value that must never be public: `JWT_SECRET`, `MONGODB_URI`, `EMAIL_PASSWORD`, `ADMIN_PASSWORD`, `DEMO_PASSWORD`.

Environment variable

Configuration supplied by the environment rather than source code, loaded from `.env` by `dotenv`.

.env is git-ignored and must never be committed. Committed secrets live in git history forever, recoverable even after a later “deletion” commit. Only `.env.example` — names, no values — is committed.

React

Component

A function returning JSX. Must start with a capital letter.

JSX

HTML-like syntax compiled to JavaScript. `className` not `class`, `htmlFor` not `for`, `{}` for expressions.

Prop

Read-only input passed from parent to child.

```
<Spinner label="Loading Jobs..." />
```

JSX

State

Data that changes and triggers a re-render.

```
const [page, setPage] = useState(1);
```

JSX

“

Never mutate state directly — always create a new value.

”

Hook

A `use...` function adding capability to a component. **Only call hooks at the top level of a component** — never inside conditions or loops, because React tracks them by call order.

HOOK	PURPOSE
<code>useState</code>	Local state
<code>useEffect</code>	Side effects after render
<code>useMemo</code>	Cache a computed value
<code>useCallback</code>	Cache a function
<code>useContext</code>	Read a context
<code>useParams</code>	Route parameters
<code>useNavigate</code>	Programmatic navigation
<code>useSearchParams</code>	Query string
<code>useLocation</code>	Current location

Context

Shares data with all descendants without prop drilling. RuWork: `AuthContext`, `ToastContext`.

Render

React calling your component to produce the UI, then updating only what changed.

Controlled input

An input whose value comes from state, with `onChange` updating that state — so state and screen can never disagree.

Conditional rendering

```
{state.status === "loading" ? <Spinner /> : null}
{state.error ? <Alert>{state.error}</Alert> : null}
```

JSX

⚠️ `{count && <Badge/>}` renders a literal 0 when count is 0. Use `count > 0 ? ... : null`.

Route

A URL-to-component mapping.

```
<Route path="/jobs/:id" element={<JobDetailsPage />} />
```

JSX

Lazy loading

Deferring a component's download until needed.

```
const AdminDashboardPage = lazy(() => import("./pages/admin/AdminDashboardPage"));
```

JSX

Suspense

A boundary that shows a fallback while a lazy component loads. Required above any lazy component.

Key

A stable identity for list items so React can update efficiently.

```
{jobs.map((job) => <JobCard key={job.id} job={job} />)}
```

JSX

“

Use a real id, not the array index — index keys cause wrong-item bugs when the list reorders.

”

Interceptor (Axios)

A hook running on every request or response. RuWork uses one to attach the token and one to handle revoked sessions.

HTTP

Methods

METHOD	MEANING	EXAMPLE
GET	Read; never changes data	GET /api/jobs
POST	Create	POST /api/jobs
PATCH	Partial update	PATCH /api/jobs/:id
DELETE	Remove	DELETE /api/jobs/:id (archives)

Status codes

CODE	NAME	MEANING	RUWORK EXAMPLE
200	OK	Success with a body	Login; job list
201	Created	A new record exists	Registration; application; review; message
400	Bad Request	Invalid input	Weak password; note under 20 chars; malformed JSON
401	Unauthorized	<i>Who are you?</i>	Missing/expired/revoked token; wrong password
403	Forbidden	<i>Not allowed.</i>	Student on an admin route; suspended account
404	Not Found	Doesn't exist (or you may not see it)	Unknown or hidden job
409	Conflict	Clashes with current state	Duplicate email; applying twice; re-approving
413	Payload Too Large	Body too big	JSON over 100 kB
429	Too Many Requests	Rate limited	11th failed login in 15 min
500	Internal Server Error	Unexpected bug	Generic message + reference id
503	Service Unavailable	Temporarily broken	Health check with DB down; verification email failed

Request/response anatomy

```

POST /api/users/login HTTP/1.1
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
```



```

{ "email": "student@ruh.ac.lk", "password": "..." }
```

HTTP

```
HTTP/1.1 200 OK
```

```
HTTP
```

```
Content-Type: application/json
```

```
{ "message": "Login successful", "token": "eyJhbGciOi..." }
```

Origin

Scheme + host + port. `http://localhost:5173` and `http://localhost:5000` are **different origins** — which is why CORS (or the Vite dev proxy) is needed.

Idempotent

Doing it twice has the same effect as once. `GET` and `DELETE` are idempotent; `POST` generally is not — which is why RuWork relies on unique indexes to stop double-submitted applications.

RuWork-specific terms

TERM	MEANING
Eligibility	The six server-side conditions a Student must satisfy for normal access (<i>Backend §5.5</i>)
Moderation	Reversible Admin action: <code>active</code> $\rightleftharpoons$ <code>suspended</code> (accounts), <code>visible</code> $\rightleftharpoons$ <code>hidden</code> (jobs), <code>active</code> $\rightleftharpoons$ <code>hidden</code> (reviews)
Archiving (Option B)	<code>DELETE /api/jobs/:id</code> sets <code>archivedAt</code> and closes the job instead of erasing it
<code>tokenVersion / tv</code>	Revocation counter; incrementing it invalidates every issued token for that account
Authoritative guard	Middleware that re-reads the account from MongoDB rather than trusting the JWT alone
Serializer	A function that picks only safe fields for a response — e.g. <code>serializeStudentProfile</code>
Aggregate (rating)	Denormalised <code>averageRating</code> + <code>reviewCount</code> stored on Job and Provider
Compensating rollback	Undoing a saved change when a follow-up step (audit write, aggregate recalculation) fails
Best-effort	An action allowed to fail silently because the core operation already succeeded — notifications
Test gate	<code>isTestEnvironment()</code> — explicit opt-in for test-only fallbacks, forced off in production
System field	A field only the server may set (<code>moderationStatus</code> , <code>jobProviderId</code> , <code>averageRating</code> , ...)
Singleton	The one-and-only <code>PlatformSetting</code> document, enforced by <code>singletonKey</code>
Participant	A Student or Provider connected by an Application — the only people who may message each other

NEXT: REQUEST & DATA FLOWS

